

Core Concepts

What the navigation manager is doing, and the settings that decide whether it works well.

~10 minutes reading. No project changes. This is the chapter that makes every later setting make sense

IN SHORT

LCM Nav3D carves your level's **empty air** into a tree of boxes (a Sparse Voxel Octree) and paths through the empty ones. You place one `LcmNavigationManagerSVO` actor per level, tell it how big your world is and how small the tightest gap is, and it does the rest.

1. Why not just use Unreal's navmesh?

Unreal's Recast navmesh is a **surface**. It answers "where can I stand?" by flattening your level onto walkable polygons. That is exactly right for a soldier and completely wrong for a dragon, because the interesting space for a flyer is the part with nothing in it.

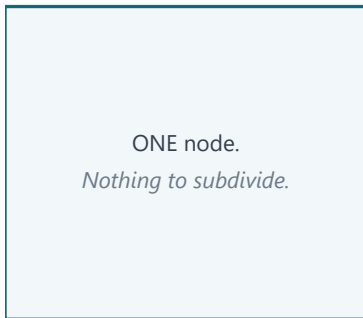
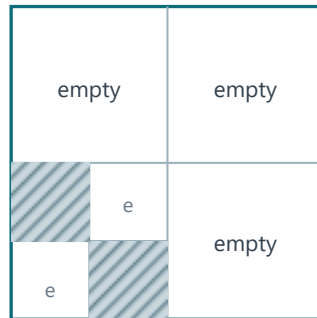
LCM Nav3D answers a different question: "**where is there empty air?**" It represents the volume, not the floor. That's the whole idea. Everything else is engineering around making that fast.

2. The Sparse Voxel Octree, in one picture

Imagine a giant cube around your level. Now:

1. Is the cube completely empty? **Yes** → done, one big box of free space.
2. **No**, something's in it? → cut it into 8 smaller cubes and ask each one the same question.
3. Repeat until the cubes reach your minimum size.

That's it. That's the octree.

EMPTY REGION**REGION CONTAINING GEOMETRY**

 solid: keeps subdividing

 empty: stops here

*Open air costs one node,
however large it is.*

Subdivision only happens where geometry is. This is why a large, mostly-open level costs far less than its size suggests.

"Sparse" is the important word. Open sky costs almost nothing: one node covers a huge volume. Detail only appears where geometry actually is. This is why a 2 km map doesn't cost 2 km worth of memory: most of it is air, and air is cheap.

What this means for you: the cost of your navigation data is driven by *how much clutter you have*, not by how big your level is.

3. The one actor you must place

`LcmNavigationManagerSVO` : drop one into your level. One per level, that's the rule.

It owns the octree, runs the pathfinder, and answers every query. Every agent talks to it. If it isn't in the level, nothing navigates.

Its Details panel is deliberately organised in the order you should think about it:

Category	What it's for
1. Setup	Finite or infinite world, the biggest decision
2. Voxels	Size and resolution
3. Collision	Which geometry counts as an obstacle
4. Performance	Frame-budget knobs
5. Debug	Visualisation
6. Runtime State	Read-only live info

Full field-by-field detail is in the [Settings Reference](#). Here we only cover the concepts behind them.

4. Finite vs Infinite: the decision that shapes everything

This is **1. Setup → Infinite World**, and it changes which other settings even appear.

Finite (default, **Infinite World** unticked)

One fixed box of navigation, built once. You set **WorldExtent** (default **10000 cm** = a 200 m cube, since extent is measured from the centre outward).

- Built at **BeginPlay** if **Auto Build On Play** is on (it is by default).
- Simple, fast, completely predictable.
- **Use this for:** arenas, levels, dungeons, single buildings, anything with a boundary.

Start here. Most projects never need anything else.

Infinite (**Infinite World** ticked)

The world is divided into **chunks** of **ChunkSize** (default **8000 cm**). Chunks are generated around your agents as they move and thrown away behind them.

- **Use this for:** open worlds, streaming levels, procedural terrain, anything without a boundary.
- Costs more complexity: chunks must generate before an agent can route through them.

NOTE

ChunkSize is not a free dial. It decides both memory *and* whether distant routes can be found at all. **8000 is the tested default. Change it only with a reason and re-test.** Larger chunks mean fewer, heavier generations; smaller means more, lighter ones, and more seams to cross.

5. Resolution: the two settings people get wrong

MinVoxelSize (default 100 cm)

How small the smallest box is allowed to get. **This is the single biggest driver of both quality and cost.**

- **Smaller** → finds tighter gaps, uses more memory, takes longer to build.
- **Larger** → cheaper and faster, but narrow openings vanish and agents route around them.

WATCH OUT It's a floor, not an exact size. The octree halves its way down, so the real leaf size is the world (or chunk) size repeatedly divided by two until it's about your `MinVoxelSize`. Two consequences that surprise people:

- Changing `MinVoxelSize` from 100 to 90 may change **nothing** (same number of halvings).
- Changing it from 100 to 60 may cross a threshold and **double your memory** in one step.

Tune it by testing, not by assuming the number is literal.

Rule of thumb: set `MinVoxelSize` to roughly **half the narrowest gap** your agents must fly through. A 3 m window needs about 150 cm or finer.

`ClearancePadding` (default 100 cm)

Inflates obstacles by this much when voxelizing, so agents don't clip walls.

WATCH OUT The classic mistake: setting this too high seals your level. If padding approaches the size of your voxels, doorways and corridors fill in completely and the pathfinder reports "no route" through an opening you can plainly see. Keep `ClearancePadding` well below `MinVoxelSize`. If you need more clearance than that, lower `MinVoxelSize` too.

If agents refuse to path through an opening, this setting is the first thing to check.

6. How a path is actually found

For a short hop, it's a straight search through the octree. For anything longer, there are two levels:

1	MACRO	which chunks / regions do I cross?	<i>coarse · cheap · long-range</i>
2	MICRO	which voxels inside them?	<i>fine · detailed · short-range</i>
3	SMOOTH	turn the blocky voxel path into a clean line	<i>validated against geometry</i>

You don't call these yourself; it's automatic. The reason to know is that it explains a common symptom: if a long route fails but a short one works, the problem is usually **macro**. Either chunks are not generated yet, or `ChunkSize` is interacting badly with your layout.

Choosing a solver

Solver	Character	Use when
A* (<i>Fastest, Grid Locked</i>)	Cheapest; paths follow voxel edges, so turns look square	You'll smooth the result anyway, or you need max throughput
Theta* (<i>Accurate, Any Angle</i>)	Most direct; cuts corners properly	Quality matters more than cost
Lazy Theta* (<i>Balanced</i>)	Nearly Theta* quality, near A* cost	The sensible default

Smoothing

Mode	Result
Raw Path	Straight out of the solver, blocky
Linear Shortcut	Removes redundant waypoints; straight segments
Curved Spline	Smooth curves: what you want for flying things

NOTE

Smoothing runs *after* pathfinding and is validated against geometry, so a smoothed path won't cut through a wall.

CPU or GPU?

Auto (the default) uses the CPU and switches to the GPU when the batch is big enough to pay off.

- **CPU** is deterministic: same input, same output, every time. Choose it explicitly if you need reproducibility (replays, lockstep multiplayer, automated tests).
- **GPU** wins on large batches of simultaneous queries.
- **No SM5 GPU? Nothing breaks.** It runs on the CPU.

Leave it on **Auto** until profiling tells you otherwise.

7. Things that move

Static geometry is baked into the octree. For things that move, add a **LcmDynamicObstacleComponent** to the actor. It restamps the octree as the actor moves, and agents reroute around it. That's [Tutorial 05](#).

8. Terms you'll meet in the rest of the docs

Term	Meaning
SVO	Sparse Voxel Octree, the navigation data
Voxel / node	One box in the octree; a leaf is a smallest one
Chunk	One tile of an infinite world
Portal	A connection between two chunks: how routes cross a seam
Macro / micro	Coarse long-range vs fine short-range pathfinding
Clearance	How much room an agent needs to fit
Flow field	One shared steering field many agents sample (for crowds)

Recap

- The octree stores **empty space**, and empty space is cheap.
- One `LcmNavigationManagerSVO` per level.
- **Finite** unless you genuinely have an open world.
- `MinVoxelSize` $\approx$ half your tightest gap; it snaps to powers of two.
- `ClearancePadding` below `MinVoxelSize`, or you'll seal your own level.
- **Lazy Theta* + Curved Spline + Auto** is a good default for a flyer.

NEXT

[Tutorial 03: Your First Flying Agent](#). Build a working agent in your own level.

Chapter 2 of the LCM Nav3D User Manual.

The complete manual is `LCM_Nav3D_Manual.pdf`, in this folder alongside every other document in the set. Start from `README.pdf`.

LCM Nav3D: True Volumetric 3D Navigation for Unreal Engine. Version 2.05 · Unreal Engine 5.8.

Copyright © 2026 L.Charitha Madhushanka. All Rights Reserved. Proprietary and confidential. Licensed, not sold; use is governed by the licence that accompanies this software. Unreal Engine is a trademark of Epic Games, Inc. All trademarks are the property of their respective owners.